

PORTADA

Análisis y Diseño de Sistemas II

Semana 5 — Estilos de Arquitectura I y II

Universidad de Antioquía · Ingeniería de Sistemas



ENCUADRE

Hoy reemplazamos el Coworking del miércoles

Esta semana el Coworking de CodeF@ctory (Planning Sprint 1) se dictó el miércoles, así que el tema que originalmente le correspondía a esa sesión se recorre y se dicta hoy, junto con el tema propio del viernes: por eso vemos **ocho estilos arquitectónicos completos** en una sola sesión.

El martes vimos qué es una decisión arquitectónica y qué atributos de calidad compromete. **Hoy vemos las opciones concretas entre las que cada equipo debe elegir para su proyecto.**

APERTURA

**De un centro de datos
a 700+ microservicios**



CASO · FICHA TÉCNICA

Netflix, 2008–2016

Sistema	Netflix — plataforma de streaming de video
Arquitectura original	Aplicación monolítica desplegada en centros de datos propios
Detonante	Agosto de 2008 — corrupción de base de datos deja el servicio caído tres días completos
Respuesta	Migración progresiva a microservicios sobre la nube pública de AWS (2009–2016)
Resultado	Más de 700 microservicios independientes, cada uno con su propio ciclo de despliegue
Fuente	Netflix Technology Blog, charlas públicas de ingeniería (ej. Adrian Cockcroft, "Migrating to Microservices")

CASO · POR QUÉ NO BASTÓ CON "ARREGLAR EL MONOLITO"

Un solo punto de falla para todo el catálogo

La caída de 2008 no fue un bug puntual: fue la consecuencia directa del **estilo arquitectónico elegido**. Un monolito desplegado en infraestructura propia significaba que un solo problema —una base de datos corrupta— podía tumbar la plataforma completa durante días.

Ningún parche de código podía resolver eso: hacía falta cambiar la estructura misma del sistema, empezando por elegir un estilo distinto.

TRANSICIÓN

El catálogo de opciones

Netflix no saltó directo de "monolito" a "microservicios" — pasó por decisiones intermedias (separar servicios, exponerlos como APIs, aislar fallos) que corresponden a varios de los estilos que existen en la disciplina.

Hoy recorreremos ocho estilos arquitectónicos, del más simple al más distribuido, para que cada equipo elija el que mejor sirva a su proyecto — no el más de moda.

CONCEPTO

Qué es un estilo arquitectónico

Un **estilo arquitectónico** es una categoría de solución estructural, reconocida y reutilizada por la industria, que define: qué tipos de componentes existen, cómo se conectan entre sí, y qué restricciones rigen esa interacción a nivel de sistema completo.

No es una tecnología ni un framework — es una forma de organizar el sistema que puede implementarse con distintas tecnologías. Elegirlo es la decisión arquitectónica más temprana y más costosa de revertir de todo el proyecto.

ESTILO 1 · MONOLÍTICA

Todo el sistema, un solo despliegue

Toda la lógica de negocio, acceso a datos e interfaz corren como **un único proceso desplegable**. Es el estilo con el que casi todo proyecto empieza — incluido Netflix en 2007 y Twitter en 2006.

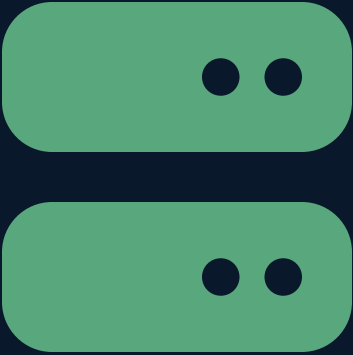
A favor	En contra
Simple de desarrollar, probar y desplegar al inicio	Un fallo en cualquier parte puede tumbar todo el sistema
Sin latencia de red entre módulos internos	Escalar significa escalar el sistema completo, no solo lo que lo necesita



Quien pide y quien responde

Divide el sistema en **clientes** (piden servicios) y **servidores** (los proveen), comunicados por red. Variantes comunes: 2 capas (cliente habla directo con la base de datos) y 3 capas (cliente, servidor de aplicación, base de datos).

A favor	En contra
Separa responsabilidades de presentación y procesamiento	El servidor puede volverse cuello de botella con muchos clientes
Varios clientes pueden compartir un mismo servidor	Depende de disponibilidad de red entre cliente y servidor



ESTILO 3 · ORIENTADA A SERVICIOS (SOA)

Servicios de negocio, reutilizables entre sistemas

Organiza el sistema como un conjunto de **servicios de negocio con contrato bien definido** (ej. "ValidarPago", "GenerarFactura"), pensados para reutilizarse entre múltiples aplicaciones de la organización, normalmente coordinados por un **bus de servicios (ESB)** centralizado.

A favor	En contra
Reutilización de servicios entre distintas aplicaciones de la empresa	El ESB centralizado puede volverse un cuello de botella y un punto único de falla
Contratos formales facilitan la integración entre sistemas heterogéneos	Gobernanza y versionado de contratos añaden complejidad operativa

ESTILO 4 · BASADA EN COMPONENTES

Piezas reemplazables con interfaz explícita

Organiza el sistema como **componentes de software independientes** —unidades de despliegue con interfaces bien definidas y dependencias explícitas— que se ensamblan para formar la aplicación completa. Cada componente puede desarrollarse, probarse y reemplazarse por separado.

A favor	En contra
Alta reutilización dentro del mismo sistema o entre proyectos	Diseñar buenas interfaces de componente exige más disciplina inicial
Reemplazar un componente no obliga a tocar los demás	Gestionar versiones de componentes puede volverse complejo



CONTRAEJEMPLO

SOA no es "lo mismo pero con otro nombre" que microservicios

Es común confundir SOA con microservicios porque ambos hablan de "servicios". **La diferencia clave está en la coordinación y el tamaño:**

- **SOA:** servicios de grano grueso (procesos de negocio completos), coordinados por un ESB centralizado, con frecuencia comparten base de datos.
- **Microservicios:** servicios de grano fino, comunicación descentralizada (sin ESB), cada uno con su propia base de datos.

No son el mismo estilo con distinto nombre de moda — resuelven problemas de coordinación de forma opuesta.

Cuando el tiempo de respuesta es parte del requisito

Organiza el sistema en **procesos o hilos que se ejecutan simultáneamente**, coordinados para cumplir restricciones temporales estrictas — no basta con que el resultado sea correcto, debe llegar **dentro de una ventana de tiempo garantizada**. Típico en sistemas embebidos, control industrial y monitoreo.

A favor	En contra
Aprovecha múltiples núcleos/procesos para responder más rápido	Condiciones de carrera y bloqueos son difíciles de depurar
Único estilo viable cuando hay plazos de tiempo estrictos (ej. control de tráfico aéreo)	Exige diseño cuidadoso de sincronización entre procesos



Un núcleo común, muchas variantes

Diseña un **núcleo de funcionalidad compartida** del cual se derivan múltiples productos o variantes, configurando qué características ("features") incluye cada uno — sin duplicar el código base. Ejemplo típico: un mismo sistema de punto de venta configurado distinto para cada tipo de cliente (retail, restaurante, farmacia).

A favor	En contra
Reduce duplicación cuando se venden varias variantes del mismo producto	Diseñar el núcleo configurable exige inversión inicial alta
Cada variante se mantiene sincronizada con las correcciones del núcleo	Solo rentable si de verdad hay múltiples variantes que mantener



Objetos como unidad estructural del sistema

Organiza el sistema como una red de **objetos que encapsulan estado y comportamiento**, colaborando entre sí mediante el envío de mensajes (llamadas a métodos). Es el estilo que subyace al modelo de clases de dominio ya construido en la Unidad 2.

A favor	En contra
Encapsulamiento facilita mantener y extender el sistema (herencia, polimorfismo)	Sin disciplina, las relaciones entre objetos pueden volverse difíciles de rastrear
Mapea naturalmente al modelo de dominio ya elaborado por cada equipo	No resuelve por sí solo cómo distribuir el sistema entre varios procesos o servidores



Servicios pequeños, independientes de punta a punta

Divide el sistema en **servicios de grano fino**, cada uno con una responsabilidad de negocio acotada, su **propia base de datos** y su propio ciclo de despliegue — sin coordinador central. Es el estilo al que Netflix migró entre 2009 y 2016.

A favor	En contra
Cada servicio escala, despliega y falla de forma independiente	Complejidad operativa alta: monitoreo, trazabilidad y consistencia entre servicios
Equipos pequeños pueden ser dueños de servicios completos (Ley de Conway a favor)	No recomendable para equipos pequeños o proyectos de corta duración



CONCEPTO · RESUMEN

Los ocho estilos, en una tabla

Estilo	Idea central
Monolítica	Un solo proceso desplegable
Cliente/Servidor	Quien pide vs. quien provee, por red
SOA	Servicios de negocio reutilizables, coordinados por un ESB
Basada en componentes	Piezas reemplazables con interfaz explícita
Concurrente / tiempo real	Procesos simultáneos con plazos de respuesta garantizados
Líneas de producto	Núcleo común, variantes configurables
Orientada a objetos	Objetos que encapsulan estado y colaboran por mensajes
Microservicios	Servicios de grano fino, independientes, sin coordinador central

TENSIÓN CONCEPTUAL

Microservicios no siempre es la respuesta correcta

Netflix necesitó ocho años y equipos dedicados de infraestructura para completar su migración a microservicios. **La complejidad operativa que ganaron —monitoreo distribuido, consistencia entre servicios, cientos de despliegues independientes— no es gratis.**

Para un proyecto de Fábrica Escuela, de equipo pequeño y duración de un semestre, un monolito bien modularizado casi siempre es la elección correcta. El estilo "de moda" no es automáticamente el mejor para el contexto propio.



CONCEPTO

Criterios para elegir, no para adivinar

- **Tamaño y experiencia del equipo:** equipos pequeños rara vez se benefician de microservicios o SOA.
- **Duración y ciclo de vida del proyecto:** un semestre no justifica la inversión de una línea de producto.
- **Atributos de calidad prioritarios** (vistos el martes): si prima la disponibilidad ante fallos parciales, un estilo distribuido puede justificarse; si prima simplicidad y velocidad de entrega, un monolito modular gana.
- **Presupuesto operativo:** más servicios independientes significa más infraestructura, monitoreo y coordinación que mantener.

INTERACCIÓN · INSUMO PARA PLANNING SPRINT 1

Elijan el estilo dominante de su proyecto

En equipos, en salas de Zoom:

1. Revisen el atributo de calidad crítico que trajeron identificado desde el martes.
2. Elijan, de los ocho estilos de hoy, el que mejor sirva a ese atributo dado su tamaño de equipo y duración del proyecto.
3. Justifiquen la elección en una frase, usando los criterios de la diapositiva anterior.

10 minutos · el resultado alimenta directamente el Planning Sprint 1 de esta semana

CONCEPTO EN DISPUTA

¿Empezar en microservicios o migrar después?

Postura "microservicios desde el día uno": evita la reescritura costosa que vivieron Twitter y Netflix — si se sabe que el sistema va a crecer, mejor diseñarlo distribuido desde el inicio.

Postura "monolito modular primero": defendida por buena parte de la industria (incluye a ingenieros de Shopify y a Martin Fowler con el término "monolito primero") — dividir un sistema que aún no se conoce bien suele producir fronteras de servicio equivocadas, más caras de corregir que empezar simple.

No hay consenso único: depende de qué tan predecible es el crecimiento y qué tan cara es la migración posterior en el contexto específico.

SÍNTESIS

Ideas clave de hoy

1. Un **estilo arquitectónico** define qué componentes existen, cómo se conectan y qué restricciones rigen esa interacción, a nivel de sistema completo.
2. Ocho estilos: monolítica, cliente/servidor, SOA, basada en componentes, concurrente/tiempo real, líneas de producto, orientada a objetos y microservicios.
3. **SOA y microservicios no son lo mismo**: difieren en grano de servicio y en coordinación centralizada vs. descentralizada.
4. Elegir un estilo depende de atributos de calidad prioritarios, tamaño de equipo y duración del proyecto — no de qué tan de moda esté.
5. No hay consenso sobre empezar distribuido o migrar después — para Fábrica Escuela, un monolito modular suele ser la opción más sensata.



CIERRE

Para la próxima semana

Lleguen al Planning Sprint 1 (Coworking de esta semana) con el **estilo arquitectónico dominante de su proyecto ya elegido y justificado** — es uno de los insumos que se revisan en esa sesión.

La semana 6 profundiza en el estilo cliente/servidor con arquitecturas N-capas, y cierra la Unidad 3 formalizando estas decisiones con diagramas UML de despliegue, paquetes y componentes.

